ZÁKLADY V C++

Kurz: **Moderní počítačová grafika**

Lektor: Ing. Michal Švento

Úvod do cvičení

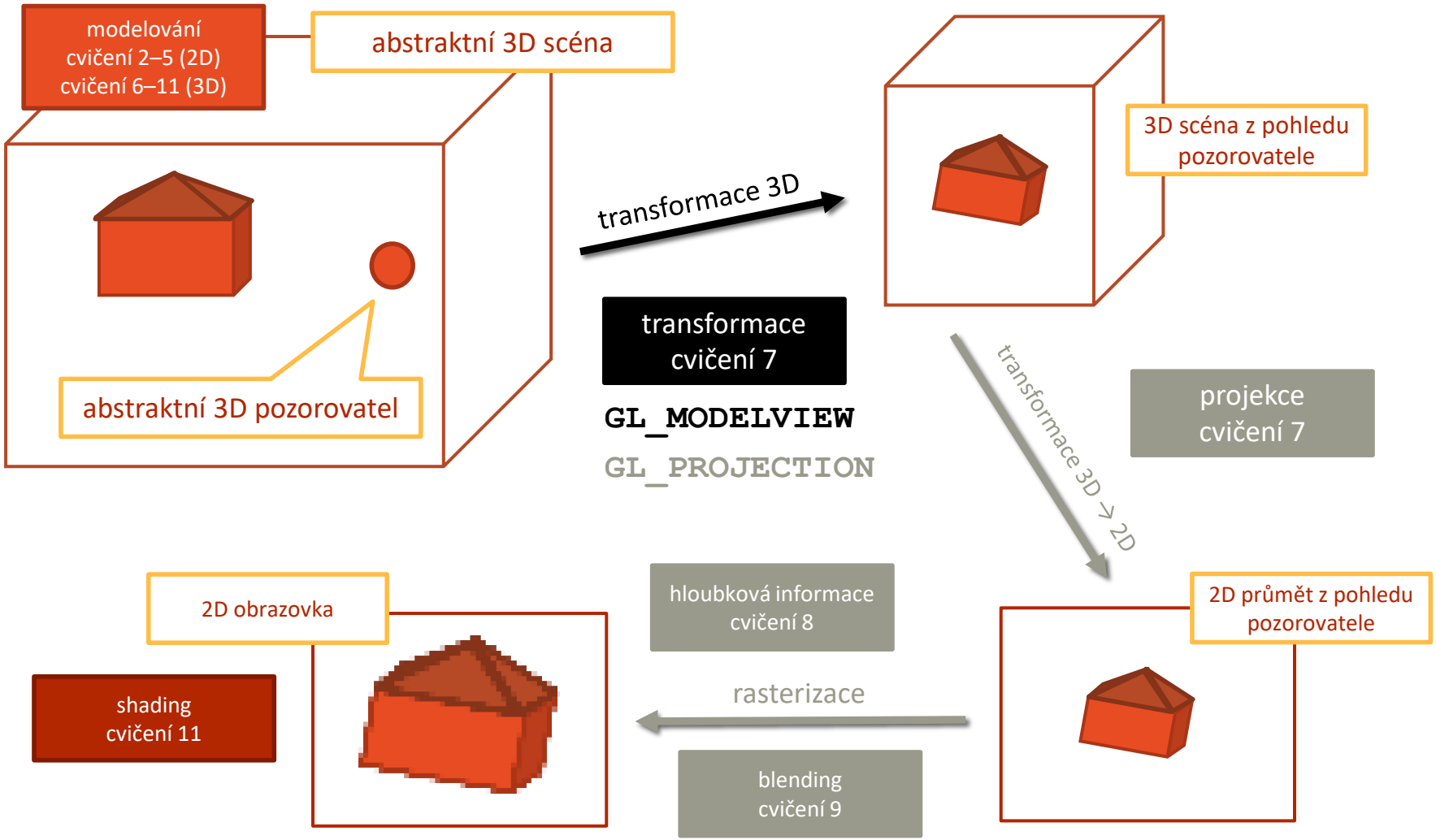
- Vyučující
  - Přednášky: **prof. doc. Mgr. Pavel Rajmic, Ph.D.** (SD5.76)
  - Cvičení: **Ing. Michal Švento** (SD5.66)
- Bodování

dva testy ve cvičení	nepovinné	max. 6 + 6 bodů
samostatné úkoly ve cvičení	nepovinné	max. 2 + 2 body
samostatný projekt	povinný, odevzdání **a schválení** před zkouškou	max. 24 bodů min. 12 bodů
písemná část zkoušky	povinná	max. 40 bodů min. 15 bodů
ústní část zkoušky	nepovinná	max. 20 bodů
zkouška celkem		max. 60 bodů min. 20 bodů
bonusové úkoly	*nepovinné*	*až 25 bodů (navíc)*
celkem		**max. 100 bodů**

Co a proč?

- Praktické vyzkoušení teoretických základů moderní počítačové grafiky

- OpenGL + GLU + GLUT
  - Rozumný kompromis mezi nízkoúrovňovým a vysokoúrovňovým
    - příliš mnoho implementace versus příliš vzdálené od teorie
  - Probrané základy se uplatňují i v současných alternativách
    - Vulkan, Unreal Engine apod.


modelování
cvičení 2–5 (2D)
cvičení 6–11 (3D)
abstraktní 3D scéna
abstraktní 3D pozorovatel
transformace 3D
3D scéna z pohledu pozorovatele
transformace
cvičení 7
GL_MODELVIEW
GL_PROJECTION
transformace 3D → 2D
projekce
cvičení 7
2D průmět z pohledu pozorovatele
hloubková informace
cvičení 8
rasterizace
blending
cvičení 9
2D obrazovka
shading
cvičení 11

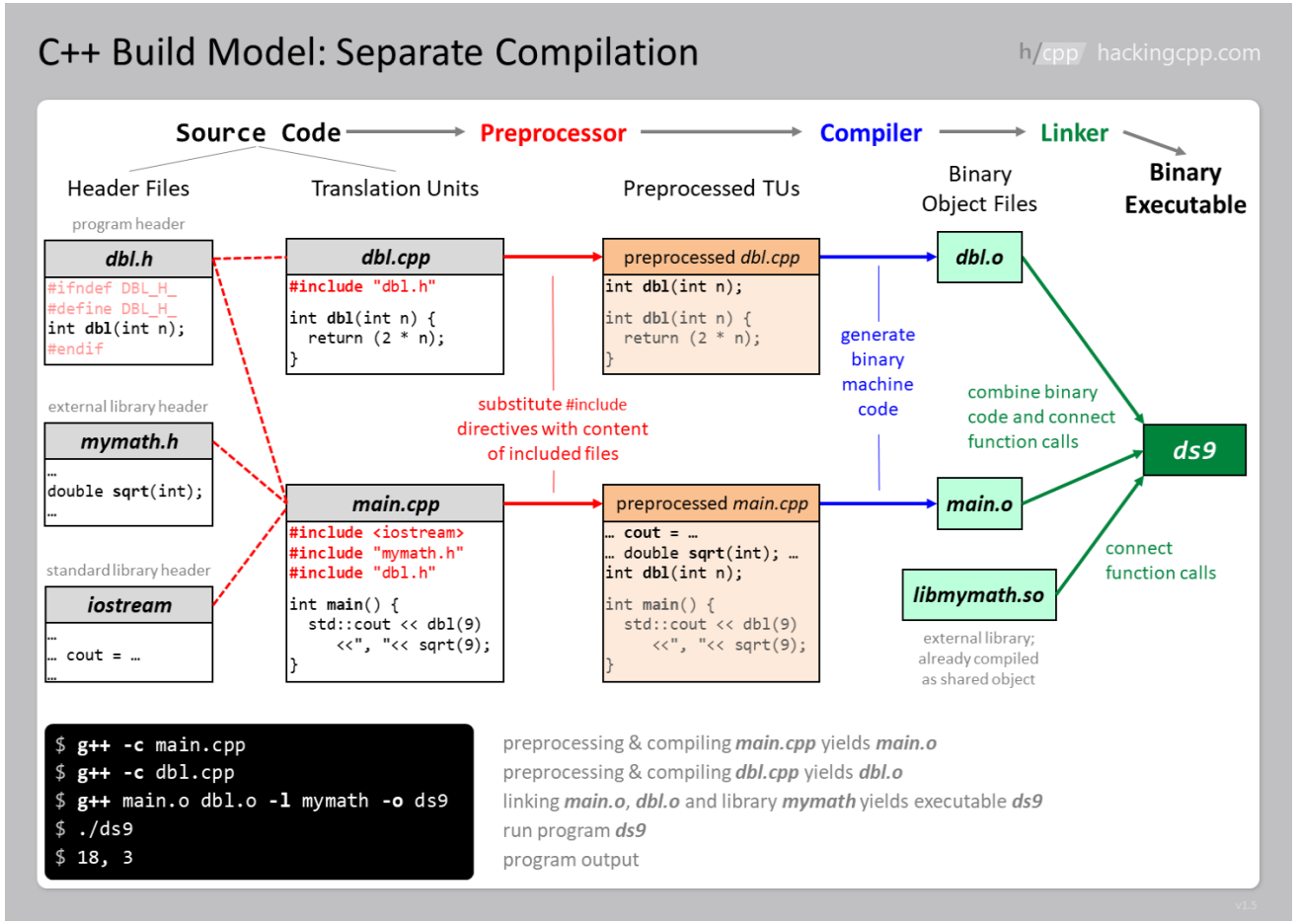
týden	Datum	Téma		Bodování	
1	9. 2. 2026	1	úvod do C++		
2	16. 2. 2026	2	2D: úvod do OpenGL, základy vykreslování		
3	23. 2. 2026	3	2D: základy vykreslování pokračování, Bézierovy křivky		
4	2. 3. 2026	4	2D: uživatelský vstup, animace		
5	9. 3. 2026	5	2D: **algoritmus de Casteljau,** Evaluátor 3D: Bézierovy pláty ve 3D, subdivisions	1. bodovaný úkol	2
6	16. 3. 2026	6	**první test** 3D: zobrazení scény, geometrické transformace	1. test	6
7	23. 3. 2026	7	3D: graf scény – dědění vlastností a transformací objektů ve 3D, pohyb ve 3D prostoru		
8	30. 3. 2026	8	3D: ořezávací algoritmy, efekty mlhy, Phongův osvětlovací model, normály		
9	6. 4. 2026	Státní svátek			
10	13. 4. 2026	9	3D: textury a práce s texturami, mipmapping, blending		
11	20. 4. 2026	10	3D: kvadriky GLU, Blender a import modelu v OpenGL	2. bodovaný úkol	2
12	27. 4. 2026	11	**druhý test** shadery	2. test	6
13	4. 5. 2026	Konzultace k projektům po předchozí domluvě			

Navíc kdykoliv před zkouškou **projekt (12–24 bodů bez bonusů)**

C ++

- Opakování základu z C/C++
  - Proměnné
  - Ukazatele
  - Podmínky, Cykly
  - Funkce
  - Struktura kódu
- Co neprojdeme, ale je nedílnou součástí C++
  - Třídy
- C++ je pro nás jen užitečným nástrojem, ale tento kurz nemá za úkol naučit programovat v C++

C++ kompilace


C++ Build Model: Separate Compilation
h/cpp hackingcpp.com
Source Code
Preprocessor
Compiler
Linker
Binary Executable
Header Files
Translation Units
Preprocessed TUs
Binary Object Files
program header
dbl.h
#ifndef DBL_H_
#define DBL_H_
int dbl(int n);
#endif
dbl.cpp
#include "dbl.h"
int dbl(int n) {
return (2 * n);
}
preprocessed dbl.cpp
int dbl(int n);
int dbl(int n) {
return (2 * n);
}
dbl.o
generate binary machine code
substitute #include directives with content of included files
combine binary code and connect function calls
ds9
external library header
mymath.h
…
double sqrt(int);
…
main.cpp
#include <iostream>
#include "mymath.h"
#include "dbl.h"
int main() {
std::cout << dbl(9)
<<", "<< sqrt(9);
}
preprocessed main.cpp
… cout = …
… double sqrt(int); …
int dbl(int n);
int main() {
std::cout << dbl(9)
<<", "<< sqrt(9);
}
main.o
connect function calls
standard library header
iostream
…
… cout = …
…
libmymath.so
external library; already compiled as shared object
$ g++ -c main.cpp
$ g++ -c dbl.cpp
$ g++ main.o dbl.o -l mymath -o ds9
$ ./ds9
$ 18, 3
preprocessing & compiling main.cpp yields main.o
preprocessing & compiling dbl.cpp yields dbl.o
linking main.o, dbl.o and library mymath yields executable ds9
run program ds9
program output
v1.5

Úkoly

1. Vypočítejte a vypište na konzoli výsledek dělení dvou proměnných
2. Změňte druhou hodnotu v poli na číslo 2 s použitím ukazatele
3. Ověřte pomocí `if`, zda je hodnota proměnné kladná a dělitelná 3
4. Vytvořte funkci, která vynásobí dvě proměnné typu `int` a vrátí jejich výsledek
5. Doplňte do funkce sum cyklus `for` pro který spočítá součet celého pole prvek po prvku
6. Vytvořte vlastní hlavičkový soubor, který bude obsahovat 2D pole. Do `main.cpp` ho naimportujte a v něm pomocí cyklu `for` spočítejte průměr pole